INFORME DE LABORATORIO

Actividad N° 01 · Semana 08

Curso: Base de Datos II

ESPEJO COSME JORCH

LAB 01

CONSOLIDACIÓN Y VERIFICACIÓN DE ESQUEMAS LÓGICOS FUNCIONALES

OBJETIVO

Organizar y validar las entidades del proyecto de Grados y Títulos en esquemas especializados, aislando responsabilidades y controlando el acceso por roles (RBAC).

FUNDAMENTO TEÓRICO

Agrupar enesquemas lógicos evita usar todo en el esquema por defecto (dbo) y permite asignar permisos por módulo de negocio, manteniendo ordenadas las bases de datos grandes.

LOS 6 ESQUEMAS DEL PROYECTO

BASE DE DATOS: GradosTitulos

- **Catalogo** — datos maestros
- **Academico** — personas y estudios
- **Tramite** — trámites y pagos
- **Evaluacion** — evaluaciones
- **Documento** — documentos
- **Seguridad** — roles y accesos

SIN ESQUEMAS
Todoqueda endbo: permisos amplios y mezclados entre módulos.

→

CON ESQUEMAS + RBAC
Cadarol recibe permisossolosobre su módulo (mínimo privilegio).

PROCEDIMIENTO PRÁCTICO

1. Elegir la BD: USE GradosTitulos
2. Verificar con IF NOT EXISTS
3. Crear cada esquema (CREATE SCHEMA)
4. Auditar en sys.schemas

```
USE GradosTitulos;
GO
IF NOT EXISTS (SELECT * FROM sys.schemas WHERE name = 'Catalogo')
    EXEC('CREATE SCHEMA Catalogo;');
-- idem: Academico, Tramite, Evaluacion, Documento, Seguridad
GO

SELECT schema_id, name AS NombreEsquema
FROM sys.schemas
WHERE name IN ('Catalogo','Academico','Tramite',
               'Evaluacion','Documento','Seguridad');
```

PREGUNTA DE VALIDACIÓN

¿De quémanera laseparación de entidades en esquemas lógicos mejora la aplicación del principio de mínimo privilegio en el control de acceso corporativo (RBAC)?

IDEACLAVE

Los permisos se otorgan por esquema: cada rol accede solo a los módulos que necesita, en lugar de recibir permisos amplios sobre todo dbo.

LAB 02

IMPLEMENTACIÓN DE ÍNDICES CLÚSTER ESTRATÉGICOS EN TABLAS PRINCIPALES

OBJETIVO

Diseñar y verificar los índices clúster en las llaves primarias de las tablas clave del proyecto, para lograr un ordenamiento físico óptimo de los registros en disco.

FUNDAMENTO TEÓRICO

Elíndiceclúster determina el orden físico en que se guardan las filas de una tabla. Una clave adecuada (como un ID autoincremental secuencial) previene la fragmentación excesiva de páginas en inserciones masivas.

ÍNDICE CLÚSTER: ORDEN FÍSICO

TABLA ORDENADA FÍSICAMENTE POR LA CLAVE CLÚSTER

Página 1	Página 2	Página 3
ID = 1	ID = 4	ID = 7
ID = 2	ID = 5	ID = 8
ID = 3	ID = 6	ID = 9

1 **único índice clúster por tabla**

CLAVE RECOMENDADA

IDautoincremental secuencial: mantiene el orden y previene fragmentación en cargas masivas.

INSERT con ID creciente

Elnuev oregistroseagregaalfinal: no hay que partir páginas y se reduce la fragmentación.

```
WHERE i.type = 1 -- CLUSTERED
```

El orden físico en disco lo define esta única clave.

PROCEDIMIENTO PRÁCTICO

1. Consultar sys.indexes
2. Unir con sys.tables y sys.schemas
3. Filtrar i.type = 1 (CLUSTERED)
4. Ordenar por esquema y tabla

```
USE GradosTitulos;
GO

SELECT s.name AS Esquema,
       t.name AS Tabla,
       i.name AS IndiceClustered,
       i.type_desc AS Tipo
FROM sys.indexes i
INNER JOIN sys.tables t ON i.object_id = t.object_id
INNER JOIN sys.schemas s ON t.schema_id = s.schema_id
WHERE i.type = 1   -- 1 indica CLUSTERED
ORDER BY s.name, t.name;
```

PREGUNTA DE VALIDACIÓN

Expliquepor quéuna tabla de base de datos relacional en SQL Server solo puede tener un único índice Clúster.

IDEACLAVE

Porque el índice clúster ES el orden físico de las filas, y los datos solo pueden estar ordenados físicamente de una única manera.

LAB 03

CREACIÓN DE ÍNDICES NO CLÚSTER CON COLUMNAS INCLUIDAS

OBJETIVO

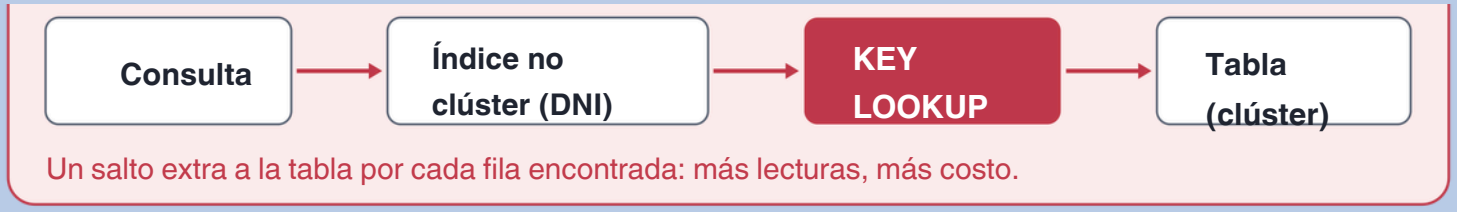
Diseñar índices No Clúster con la cláusula INCLUDE para evitar búsquedas costosas de tipo Key Lookup en consultas frecuentes de padrones y reportes.

FUNDAMENTO TEÓRICO

Losíndicesno clúster guardan las columnas clave y punteros a los datos. Con INCLUDE se añaden columnas no clave en las hojas del índice, logrando consultas totalmente cubiertas (covering indexes) de alto rendimiento.

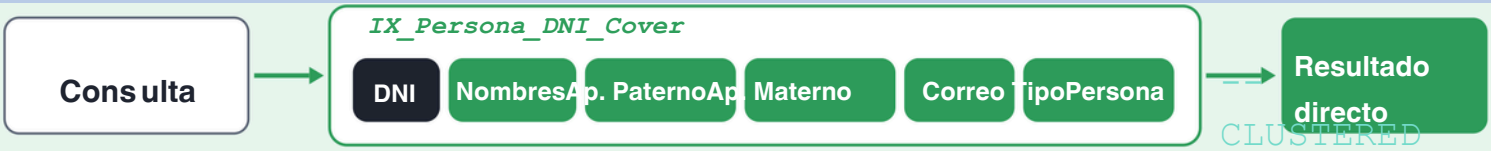
KEY LOOKUP vs. ÍNDICE CUBIERTO

SIN INCLUDE · Key Lookup



Un salto extra a la tabla por cada fila encontrada: más lecturas, más costo.

CON INCLUDE· Índice cubierto



Todas las columnas se leen desde las hojas del propio índice (sin tocar la tabla).

PROCEDIMIENTO PRÁCTICO

1. Verificar que el índice no exista
2. CREATE NONCLUSTERED sobre Persona (DNI)
3. INCLUDE de las columnas a leer
4. Repetir para TipoPersona

```
IF NOT EXISTS (SELECT * FROM sys.indexes
    WHERE name = 'IX_Persona_DNI_Cover'
    AND object_id = OBJECT_ID('Academico.Persona'))
BEGIN
    CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
    ON Academico.Persona (DNI)
    INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno,
             Correo, TipoPersona);
END;
-- 2.° índice: IX_Persona_TipoPersona ON (TipoPersona)

--   INCLUDE (DNI, Nombres, ApellidoPaterno, ApellidoMaterno)
```

PRE GUNTA DE VALIDACIÓN

¿Quédiferenciaderendimiento existe entre una consulta que utiliza un índice cubierto con INCLUDE frente a otra que requiere un operador de Key Lookup?

IDEACLAVE

Con INCLUDE todo se lee del propio índice; con Key Lookup se hace un salto extra a la tabla por cada fila, lo que implica más lecturas (I/O) y más tiempo.

LAB 04

INDEXACIÓN COMPUESTA PARA CONSULTAS ANALÍTICAS DE TRÁMITES

OBJETIVO

Construir índices compuestos no clúster según los patrones de filtrado por rangos de fechas y estados administrativos, para agilizar los reportes gerenciales.

FUNDAMENTO TEÓRICO

En consultas analíticas quecombinan igualdad (ej. estado del trámite) y rangos (ej. fecha de inicio), el orden de las columnas dentro de un índice compuesto es crítico para que el optimizador de SQL Server lo aproveche.

ANATOMÍA DE LOS ÍNDICES COMPUESTOS

IX_Tramite_Estado_Fecha

ONTramite.Tramite

COLUMNAS CLAVE (el orden importa)

1. IdEstadoTramite — igualdad (=)
2. FechaInicio — rango de fechas

INCLUDE (solo para cubrir la consulta)

IdMatricula · IdTipoTramite · IdCoordinacion

IX_Pago_Tramite_Fecha

ON Tramite.Pago

COLUMNAS CLAVE (el orden importa)

1. IdTramite — igualdad (=)
2. FechaPago — rango de fechas

INCLUDE (solo para cubrir la consulta)

Monto · NroRecibo · TipoPago

ANALOGÍA: una guía telefónica ordenada por Apellido y luego por Nombre.

Filtro por ColumnaA (y luego B): búsqueda directa al rango.

Filtro solo por ColumnaB:nosepuede saltar directo, hay querecorrer.

PROCEDIMIENTO PRÁCTICO

1. **Separar columnas de igualdad y de rango**
2. **Orden: igualdad primero, rango después**
3. **Agregar INCLUDE para cubrir**
4. **Crear índices de Trámite y Pago**

```
USE GradosTitulos;
GO

-- Reportes de trámites por Estado y rango de Fechas
CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
ON   Tramite.Tramite   (IdEstadoTramite,   FechaInicio)
INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);

-- Consulta de pagos por trámite y fecha
CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
ON Tramite.Pago (IdTramite, FechaPago)
INCLUDE (Monto, NroRecibo, TipoPago);
```

PREGUNTA DE VALIDACIÓN

¿Por qué el orden de las columnas en un índice compuesto (ColumnaA, ColumnaB) afecta directamente la capacidad de búsqueda del optimizador cuando el filtro solo utiliza ColumnaB?

IDEACLAVE

El índice está ordenado primero por ColumnaA y luego por ColumnaB dentro de cada valor de A; sin filtrar por A no se puede ir directo al rango de B.

LAB 05

AUDITORÍA, MONITOREO Y ANÁLISIS DE FRAGMENTACIÓN DE ÍNDICES

OBJETIVO

Evaluar elestado operativo de los índices: detectar los duplicados o sin uso y medir el nivel de fragmentación de las estructuras de indexación del proyecto.

FUNDAMENTO TEÓRICO

Conoperacionesintensivasde inserción, actualización y borrado (DML), los índices sufren fragmentación interna. Monitorear su estado permite aplicar mantenimiento: reorganizar o reconstruir.

QUÉ AUDITAMOS Y CUÁNDO ACTUAR

1. **Duplicados** — Índices repetidos sobre las mismas columnas.
2. **Sin uso** — Índices que nadie consulta pero se mantienen.
3. **Fragmentados** — Páginas desordenadas tras INSERT, UPDATE y DELETE.

% DE FRAGMENTACIÓN (avg_fragmentation_in_percent)

filtro del script: > 10 %

0 – 5 %	5 – 30 %	más de 30 %
Estado sano Sin acciónde mantenimiento.	**REORGANIZE** ALTERINDEX...REORGANIZE: mantenimiento ligero del índice.	**REBUILD** ALTERINDEX ... REBUILD: reconstrucción completa del índice.

Umbrales de referencia de Microsoft. La guía lista los índices con más de 10 % de fragmentación junto con su cantidad de páginas (page_count).

PROCEDIMIENTO PRÁCTICO

1. **Inventario de índices (sys.indexes)**
2. **Medir la fragmentación física del índice**
3. **Filtrar fragmentación mayor a 10 %**
4. **Decidir: REORGANIZE o REBUILD**

```
-- 1. Inventario de índices por esquema y tabla
SELECT s.name AS Esquema, t.name AS Tabla,
       i.name AS NombreIndice, i.type_desc AS TipoIndice,
       i.is_primary_key, i.is_unique
FROM sys.indexes i -- JOIN sys.tables / sys.schemas
WHERE i.name IS NOT NULL;

-- 2. Fragmentación física
SELECT ps.avg_fragmentation_in_percent, ps.page_count
FROM sys.dm_db_index_physical_stats
     (DB_ID(), NULL, NULL, NULL, 'LIMITED') ps
WHERE ps.avg_fragmentation_in_percent > 10;
```

PREGUNTA DE VALIDACIÓN

¿Cuál es el umbral de fragmentación recomendado en SQL Server a partir del cual se debe evaluar la reconstrucción (REBUILD) frente a una simple reorganización (REORGANIZE) de un índice?

IDEACLAVE

Referencia habitual de Microsoft: entre 5 % y 30 % conviene REORGANIZE; por encima de 30 % se evalúa REBUILD.